



VERIQTA

From Basics
to
Production

LINUX PRODUCTION HANDBOOK

Commands • Administration • Monitoring • Troubleshooting
Best Practices • Real-World Skills



COMMANDS



SYSTEM ADMIN



SECURITY



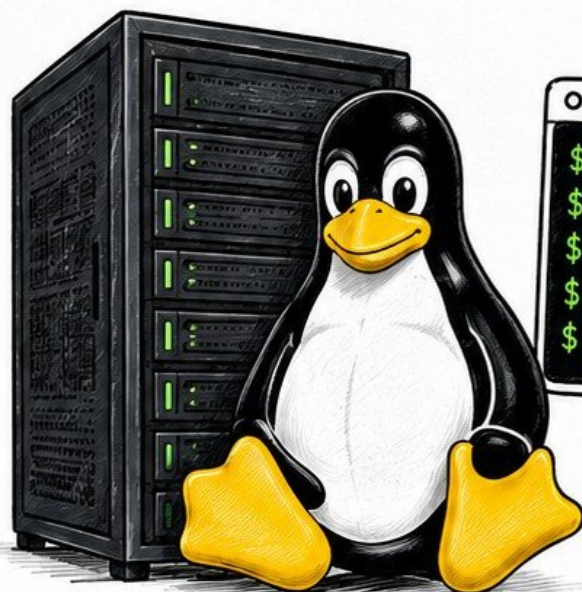
MONITORING



TROUBLESHOOTING



PRODUCTION
BEST PRACTICES



```
$ ls -l
$ top
$ systemctl status
$ df -h
$ journalctl -xe
```



LEARN



CONFIGURE



TROUBLESHOOT



MONITOR



SECURE



DEPLOY



Build. Secure. Monitor. Scale. That's **Linux**.

1. INTRODUCTION TO LINUX

Linux is more than an operating system. It is the foundation that powers the modern internet and cloud.

WHAT IS LINUX?



Linux is an open source, Unix-like operating system kernel created by Linus Torvalds in 1991.

It is the core (kernel) that manages hardware and resources.

Combined with tools, libraries and applications, it becomes a complete operating system (OS).

WHY LINUX DOMINATES SERVERS & CLOUD

- ✓ Open Source & Free - No license cost, community driven.
- ✓ Stable & Reliable - Built for uptime and long running systems.
- ✓ Secure by Design - Strong permission model and active security community.
- ✓ High Performance - Lightweight and efficient resource usage.
- ✓ Flexible & Customizable - Supports everything from small devices to massive cloud infrastructure.
- ✓ Cloud Native - Powers containers, Kubernetes, DevOps and microservices.
- ✓ Industry Standard - Used by major companies, cloud providers and data centers worldwide.

POPULAR LINUX DISTRIBUTIONS



Ubuntu - User friendly, great community support. Popular for servers and desktops.



CentOS Stream / Rocky Linux / AlmaLinux - Enterprise grade, stable and production ready.



Red Hat Enterprise Linux (RHEL) - Enterprise support and reliability.



openSUSE - Enterprise and desktop focused.



Debian - Very stable, secure and widely used.



Arch Linux - Lightweight, rolling release for advanced users.

WHERE LINUX IS USED IN PRODUCTION



Web Servers
(Apache, NGINX)



Databases
(MySQL, PostgreSQL, MongoDB)



Containers & Kubernetes



Cloud Platforms
(AWS, Azure, GCP)



File & Storage
Servers



Firewall & Security
Systems

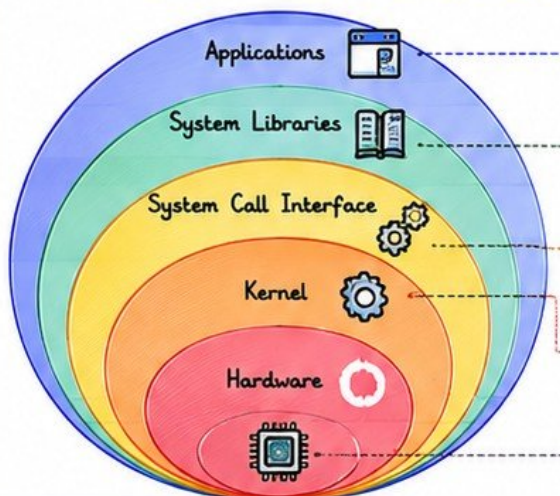


Monitoring & Logging
(Prometheus, ELK)



DevOps & CI/CD
Pipelines

LINUX ARCHITECTURE OVERVIEW



APPLICATIONS

User applications and programs that you run in Linux. Examples: Web servers, databases, scripts, Docker, etc.



SYSTEM LIBRARIES

Standard libraries (glibc, SSL, etc.) that applications use to interact with the system.



SYSTEM CALL INTERFACE

Provides an interface between applications and the kernel through system calls.



KERNEL

The core of Linux. Manages CPU, Memory, Processes, File System, Networking, and Device Drivers.



HARDWARE

Physical components such as CPU, RAM, Disk, Network Cards and other devices.



Linux is the backbone of modern IT infrastructure, powering the internet, cloud and everything in between.

VERIQTA

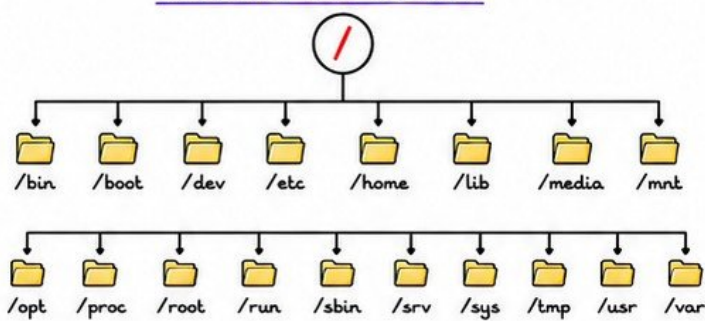
2. LINUX FILE SYSTEM

Everything in Linux is a **file**, even devices, processes and network connections.

Linux organizes everything in a single hierarchical tree structure, starting from the **root (/)**

DIRECTORY	PURPOSE	PRODUCTION EXAMPLES
 / (Root)	The top of the filesystem hierarchy. Everything starts here.	- All directories are under / - Full access required (root privileges) 
 /etc	Configuration files for the system and installed software.	/etc/nginx/nginx.conf /etc/ssh/sshd_config 
 /var	Variable data that changes during system operation (logs, caches, spools, databases).	/var/log/syslog /var/lib/mysql (database files) /var/cache/apt/archives 
 /home	Home directories for regular users. Each user has a subdirectory.	/home/ubuntu/ /home/deploy/app/ 
 /usr	User programs, utilities, libraries and documentation. Mostly read-only.	/usr/bin (user commands) /usr/lib (libraries) /usr/share (shared data) 
 /opt	Optional / third-party software installed by administrators.	/opt/docker/ /opt/jdk/ 
 /tmp	Temporary files. Cleared on reboot (in most distributions). Used for short-term storage.	/tmp/app.sock /tmp/upload/ 

FILESYSTEM TREE OVERVIEW



Some directories like **/bin**, **/sbin**, **/lib**, **/dev**, **/proc**, **/sys**, **/run** are essential for system boot and operation.

PRODUCTION EXAMPLES

- ✓ Logs: **/var/log/** contains logs for troubleshooting.
- ✓ Configurations: **/etc** holds configs for all services.
- ✓ User data: Apps store user uploads in **/home** or **/var**.
- ✓ Applications: Installed software files live in **/usr** or **/opt**.
- ✓ Temporary files: Apps create temp files in **/tmp**.
- ✓ Backups: Often stored in **/opt/backups** or **/var/backups**.
- ✓ Databases: MySQL data in **/var/lib/mysql**.
- ✓ Web servers: Website files in **/var/www/html**.
- ✓ Security: SSH configs in **/etc/ssh/**.

★ Know your filesystem. It is the foundation of Linux administration, security and troubleshooting. When you understand where things live, you can manage systems with confidence!



VERIQTA

3. NAVIGATING LINUX

Efficient navigation
is the first skill
every Linux engineer
must master.

PWD - PRINT WORKING DIRECTORY



Know where
you are.

Shows your current location in
the filesystem.

```
$ pwd
/home/ubuntu/projects/app
```

LS - LIST

Lists files and directories.

- ls
- ls -l (long format)
- ls -a (show hidden files)
- ls -lah (human readable)

```
$ ls -lah
total 32K
drwxr-xr-x 4 ubuntu ubuntu 4.0K May 10 app
drwxr-xr-x 3 ubuntu ubuntu 4.0K May 10 docs
-rw-r--r-- 1 ubuntu ubuntu 1.2K May 10 readme.md
```

CD - CHANGE DIRECTORY



Move around
the filesystem.

Moves you between directories.

- cd /path/to/dir (absolute)
- cd dir (relative)
- cd .. (go up one level)
- cd ~ or cd (go to home)

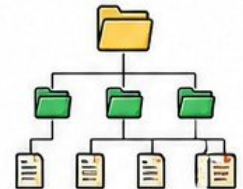
```
$ cd /var/log
$ pwd
/var/log
```

TREE - DIRECTORY TREE

Displays directory structure in a tree format.

- tree
- tree -L 2 (limit depth)
- tree -a (include hidden files)

```
$ tree -L 2 /etc
/etc
├── nginx
│   ├── nginx.conf
│   └── conf.d
├── ssh
│   └── sshd_config
└── hosts
3 directories, 3 files
```



Visualize directory
structure.

FIND - SEARCH FOR FILES



Powerful search
tool.

Find files and directories by name,
type, size, time and more.

- find /path -name "file.txt"
- find /var/log -type f -name "*.log"
- find / -type d -name "old" 2>/dev/null

```
$ find /var/log -type f -name "*.log"
/var/log/syslog
/var/log/auth.log
/var/log/nginx/access.log
```

LOCATE - QUICK FILE LOOKUP

Search for files using an index database.
(Much faster than find)

- locate filename
- locate -i filename (case insensitive)
- sudo updatedb (update database)

```
$ locate nginx.conf
/etc/nginx/nginx.conf
/usr/share/doc/nginx/examples/nginx.conf
```



Super fast
file lookup.

✓ BEST PRACTICES



- ✓ Use tab completion to save time. **Ctrl +**
- ✓ **Ctrl + I** to reverse search in history.
- ✓ Use absolute paths in scripts and automation.
- ✓ Regularly clean the locate database.
sudo updatedb
- ✓ Stay organized: Use clear directory names.
- ✓ Use ls -lah to understand permissions and sizes.
- ✓ Use man <command> to learn more.

✗ COMMON MISTAKES

- ✗ Running commands in the wrong directory.
→ Always check with pwd.
- ✗ Using rm -rf / **(DESTRUCTIVE!)**
- ✗ Forgetting to use sudo for privileged operations.
- ✗ Not reading command output or error messages.
- ✗ Using relative paths in scripts (can break).
- ✗ Not verifying before deleting or overwriting files.



REMEMBER:

The filesystem is like a map.
The better you navigate,
the faster you solve problems.



TIP: Great Linux engineers are not those who memorize commands,
but those who understand the filesystem and build good habits.



4. WORKING WITH FILES & DIRECTORIES



TOUCH - CREATE FILES

Creates an empty file or updates timestamp if the file exists.

```
$ touch file.txt
$ touch app.log
```



MKDIR - CREATE DIRECTORIES

Creates a new directory.

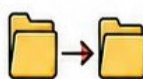
```
$ mkdir mydir
$ mkdir -p /opt/app/logs/data
```



CP - COPY FILES & DIRECTORIES

Copies files or directories.

```
$ cp file.txt /backup/
$ cp -r /etc/nginx /backup/nginx
```



MV - MOVE OR RENAME

Moves a file/directory or renames it.

```
$ mv oldname.txt newname.txt
$ mv /tmp/data /var/data
```



RM - REMOVE FILES & DIRECTORIES

Deletes files or directories.

```
$ rm file.txt          # remove file
$ rm -r olddir/        # remove directory
```



LN - CREATE LINKS

Creates links to files or directories.

```
$ ln source.txt link.txt          # hard link
$ ln -s /var/log/app.log app.log  # symbolic link
```

SYMBOLIC LINKS VS HARD LINKS

Directory

file1.txt (inode: 1234)
file2.txt (inode: 1234)



HARD LINK

- Points directly to the inode (data).
- Multiple names for the same data.
- Works only within the same filesystem.
- Original file must exist.
- Deleting one link does not delete data.

SYMBOLIC LINK (SOFT LINK)

- Points to the path of the file.
- Can point across filesystems.
- If original file is deleted, link is broken.
- Acts like a shortcut.
- Identified by file type "l".

Directory

original.txt (inode: 5678)
link.txt -> original.txt



Path reference to original.txt

PRODUCTION USE CASES

USE CASE	COMMAND/TECHNIQUE	EXAMPLE	WHY IT MATTERS
Create log files for applications	<code>touch</code>	<code>\$ touch /var/log/myapp.log</code>	Pre-create log files with correct permissions before app starts.
Create application directories	<code>mkdir -p</code>	<code>\$ mkdir -p /opt/myapp/{bin,conf,logs,data}</code>	Keeps directory structure clean and organized.
Backup configuration or data	<code>cp / cp -r</code>	<code>\$ cp -r /etc/nginx /backup/nginx_\$(date +%F)</code>	Fast way to copy configs or data before making changes.
Move files to archive or new location	<code>mv</code>	<code>\$ mv /tmp/report.txt /data/archive/</code>	Organizes files and manages storage efficiently.
Remove old logs temporarily	<code>rm</code>	<code>\$ rm /var/log/app.log.1</code>	Frees up disk space and keeps systems healthy.
Use links for shared files or shortcuts	<code>ln / ln -s</code>	<code>\$ ln -s /var/log/app.log /opt/app/log/current.log</code>	Provide stable paths for apps even if logs rotate or move.



Always double-check paths before running **rm** or **mv**. One wrong command can cause real damage!



VERIQTA

5. VIEWING & EDITING FILES

Know your files.
Read them, understand
them, edit them safely.
That's how you solve
real problems in production.

VIEWING FILES

CAT - DISPLAY FILE CONTENTS



Displays entire file content.
Best for small files.

```
$ cat /etc/hosts
```

USE WHEN

File is small
(few lines)
or you want
full output.

LESS - VIEW FILE PAGE BY PAGE



Allows you to scroll up and down.
Best for large files.

```
$ less /var/log/syslog
```

LESS SHORTCUTS

↑/↓ Scroll
PageUp/PageDown
Space Next page
b Back one page
g Go to top
G Go to bottom
/text Search
q Quit

MORE - VIEW FILE PAGE BY PAGE (FORWARD ONLY)



Older tool than less.
Scrolls forward only.
Press q to quit.

```
$ more /var/log/dmesg
```

USE WHEN

You only need
forward scrolling
(rarely used
nowadays).

HEAD - SHOW FIRST N LINES



Displays first 10 lines by default.
Good for quick preview.

```
$ head /var/log/auth.log  
$ head -n 20 largefile.log
```

USE WHEN

Check file
beginning or
headers.

TAIL - SHOW LAST N LINES



Displays last 10 lines by default.
Very useful for logs.

```
$ tail /var/log/syslog  
$ tail -n 50 app.log
```

TAIL OPTIONS

-f Follow (real-time)
-n Last N lines
-f Great for live
logs monitoring

★ TIP

For real-time log monitoring:

```
$ tail -f /var/log/nginx/access.log
```



EDITING FILES

NANO - SIMPLE TEXT EDITOR



Easy to use. Great for quick edits.

```
$ nano /etc/nginx/nginx.conf
```

NANO SHORTCUTS

Ctrl + O	Save	Ctrl + W	Search
Ctrl + X	Exit	Ctrl + \	Replace
Ctrl + K	Cut line	Ctrl + G	Help
Ctrl + U	Paste line		



VIM - POWERFUL TEXT EDITOR

Powerful and available everywhere.
Has learning curve, but worth it!

```
$ vim /etc/fstab
```

VIM BASIC WORKFLOW

- | | | |
|---|--------------|------------------------------|
| 1 | vim filename | → opens file in Normal mode |
| 2 | i | → insert mode (start typing) |
| 3 | Esc | → back to Normal mode |
| 4 | :w | → write (save) |
| 5 | :q | → quit |
| 6 | :wq | → write and quit |
| 7 | :q! | → quit without saving |

USE WHEN

Editing config
files, scripts,
code.
Essential skill
for every Linux
engineer.

💡 Learn Vim.
Use it.
Master it.

REAL PRODUCTION WORKFLOW - READING & EDITING FILES SAFELY



1. Locate the file

```
find / -name "*.conf" 2>/dev/null
```



2. Preview the file

```
head -n 20 file.conf
```



3. Read full content

```
less file.conf
```



4. Edit the file

```
nano file.conf
```


or

```
vim file.conf
```



5. Save changes



6. Verify the change

```
cat file.conf
```


or

```
systemctl status service
```

✓ BEST PRACTICES

- Always preview before editing.
- Take backup before changing important files.
- Use less for large files, not cat.
- Use tail -f for live logs.
- Exit editors properly (save or discard intentionally).

★ REMEMBER

In production, wrong edits
can break services.
Read carefully.
Edit responsibly.
Test before you restart!



✗ COMMON MISTAKES

- Using cat on huge log files (floods terminal).
- Editing without backup.
- Saving with wrong permissions.
- Quitting vim without saving (:q!).
- Forgetting to check service after edits.

VERIQTA

6. LINUX PERMISSIONS

Permissions protect your system. Set the right access, give least privilege and stay secure.



WHO CAN ACCESS A FILE OR DIRECTORY?



USERS (OWNER)

The owner of the file/directory.
Usually the creator.



GROUPS

Members of the group that owns the file.



OTHERS

Everyone else on the system.

PERMISSION TYPES

r (read) - view content
w (write) - modify content
y (execute) - run file or enter directory



PERMISSION BITS (BASIC)

Permission	Symbol	Meaning	For Files	For Directories
Read	r	View content	View file content	List files in directory
Write	w	Modify content	Edit file content	Create, delete, rename files in directory
Execute	x	Execute / Run	Run file as program	Enter / access directory

HOW PERMISSIONS LOOK

-rwxr-xr-- file.txt
 User (Owner) Group Others

Total = 9 characters (except first -)
 1st char : File type (- file, d directory)
 Next 9 : Permissions (rwx for each category)

CHMOD - CHANGE PERMISSIONS

Change read/write/execute permissions.



```
$ chmod 755 script.sh
$ chmod u+x script.sh
$ chmod go-w file.txt
$ chmod -R 755 /var/www/html
```

CHOWN - CHANGE OWNER

Change the owner of a file/directory.



```
$ chown user file.txt
$ chown -R user:group /opt/app
```

CHGRP - CHANGE GROUP

Change the group of a file/directory.



```
$ chgrp developers file.txt
$ chgrp -R webteam /var/www
```

NUMERIC (OCTAL) VS SYMBOLIC PERMISSIONS

NUMERIC (OCTAL) PERMISSIONS

Each permission has a value:

r	w	x
4	2	1

Add the values to get the number.

Permission	Binary	Number	Example
rwx	111	7	Full access
rw-	110	6	Read & Write
r-x	101	5	Read & Execute
r--	100	4	Read only
-wx	011	3	Write & Execute
-w-	010	2	Write only
--x	001	1	Execute only
---	000	0	No access

EXAMPLE

rwx r-x r--

7 5 4

= 754

So: `chmod 754 file.txt`

SYMBOLIC PERMISSIONS

Use letters (u, g, o, a) with operators (+, -, =)

Who	Letter	Meaning
u	user	Owner
g	group	Group
o	other	Others
a	all	All (u + g + o)

Operator	Meaning	Example
+	Add permission	u+x (add execute to user)
-	Remove permission	go-w (remove write from group & others)
=	Set exact permission	u=rw,g=r,o= (set exact)

SECURITY BEST PRACTICES



- Use least privilege: Give only the access required.
- Do not run everything as root.
- Use groups to manage access easily.
- Use 750 for directories and 640 for files by default.

- Avoid 777 (world full access). Very risky!
- Protect sensitive files: 600 (owner read/write).
- Use sudo instead of switching to root.
- Regularly review permissions and ownership.

- Use ACLs for advanced access control.
- Monitor critical files: /etc/passwd, /etc/shadow, /var/log, /etc/sudoers.
- Backup permission changes in production.



TIP: Before deploying an app, always verify:

Who owns the files? Who can read them? Who can modify them? Who can execute them?

REMEMBER

Good permissions = Security
 Bad permissions = Breach



VERIQTA

7. USERS & GROUPS

Strong access control starts with the right users, the right groups and the right privileges.



Linux is a multi-user system. Users own files, run processes and have permissions. Groups help manage access for multiple users.



USER MANAGEMENT COMMANDS

USERADD - CREATE USER



Creates a new user.

```
$ sudo useradd alice
$ sudo useradd -m bob
$ sudo useradd -m -s /bin/bash carol
```

Common Options

- m Create home directory
- d Specify home directory
- s Set default shell
- u Set user ID (UID)
- e Set expiry date

PASSWD - SET PASSWORD



Sets or changes user password.

```
$ sudo passwd alice
$ sudo passwd bob
```

Best Practice

Use strong passwords and change regularly.

USERMOD - MODIFY USER



Modifies existing user account.

```
$ sudo usermod -aG devops alice
$ sudo usermod -s /bin/zsh bob
$ sudo usermod -l newname oldname
$ sudo usermod -d /home/newdir bob
```

Common Options

- aG Add user to a group
- s Change default shell
- l Change username
- d Change home directory

GROUPADD - CREATE GROUP



Creates a new group.

```
$ sudo groupadd devops
$ sudo groupadd ops
```

Common Options

- g Specify GID (Group ID)

MANAGING GROUP MEMBERSHIP



- Add user to group:

```
$ sudo usermod -aG devops alice
```

- Remove user from group:

```
$ sudo gpasswd -d alice devops
```

- List groups for a user:

```
$ groups alice
```

- List all groups:

```
$ cat /etc/group
```

SUDO - SUPERUSER DO



Allows authorized users to run commands as another user (usually root).

```
$ sudo command
$ sudo -i
$ sudo su -
```

Configure sudo access

- Edit sudoers file safely:

```
sudo visudo
```

- Add user with limited privileges:

```
alice ALL=(root) /usr/bin/systemctl restart nginx
```



Never edit /etc/sudoers with a normal editor. Always use visudo.

PRINCIPLE OF LEAST PRIVILEGE (PoLP)



Give users only the minimum access they need to do their job. Nothing more.

Benefits

- ✓ Reduces risk of accidental damage
- ✓ Limits the impact of compromised accounts
- ✓ Improves accountability and traceability
- ✓ Strengthens overall security posture

★ Golden Rule

If a task can be done without root, don't use root.

PRODUCTION EXAMPLES

SCENARIO	ACTION	COMMANDS	WHY
 Create a developer user	Create user and add to dev group	<pre>\$ sudo useradd -m -s /bin/bash dev1 \$ sudo groupadd dev \$ sudo usermod -aG dev dev1</pre>	Developers work without root but can access project files.
 Deploy application as dedicated user	Create system user without login shell	<pre>\$ sudo useradd -r -s /usr/sbin/nologin appuser</pre>	Application runs with limited permissions, safer.
 Restrict access to sensitive directory	Create group and set owner and permissions	<pre>\$ sudo groupadd finance \$ sudo chown -R root:finance /data/finance \$ sudo chmod 750 /data/finance</pre>	Only finance group can access the directory.
 Allow user to restart nginx only	Give sudo access for a single command	<pre>\$ alice ALL=(root) /usr/bin/systemctl restart nginx</pre>	Grants specific privilege, not full root access.



TIP: Regularly review users and groups:

```
$ cat /etc/passwd $ cat /etc/group $ id username
```

Remove inactive accounts and unused privileges.



THINGS TO AVOID

- ✗ Running applications as root
- ✗ Sharing root password
- ✗ Adding users to sudo without reason

Good user & group management is the foundation of a secure and maintainable system.



VERIQTA

8. PROCESSES

A process is a running instance of a program.
Linux is a multi-tasking OS: many processes run at the same time.



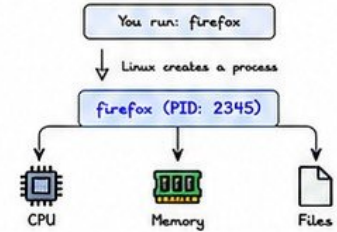
WHAT IS A PROCESS?

A process is an instance of a program in execution. Each process has a PID (Process ID), owner, state, resources (CPU, memory, files) and priority.

PROCESS ATTRIBUTES

- ✓ PID - Unique Process ID
- ✓ PPID - Parent Process ID
- ✓ USER - Owner
- ✓ STATE - Running / Sleeping / Stopped / Zombie
- ✓ %CPU - CPU usage
- ✓ %MEM - Memory usage
- ✓ START - When it started
- ✓ COMMAND - The program/command

PROCESS EXAMPLE



ESSENTIAL PROCESS COMMANDS



PS - SNAPSHOT OF PROCESSES

Shows current running processes.

`$ ps aux`

USER	PID	%CPU	%MEM	VSZ	STAT	COMMAND
root	1	0.0	0.1	2256	Ss	systemd
nginx	1234	0.1	1.2	123456	S	nginx
alice	2345	2.5	3.1	234567	Sl	firefox

Useful Options

- `ps aux` # all processes, all users
- `ps -ef` # full format
- `ps -u alice` # processes of user alice
- `ps -p 1234` # show process with PID 1234
- `ps -C nginx` # show processes named nginx



TOP - REAL-TIME VIEW

Shows live system processes.

`top - 11:25:30 up 2 days, 3:14, 2 users, load average: 0.45, 0.32, 0.28`
Tasks: 187 total, 1 running, 186 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.0 us, 1.0 sy, 0.0 ni, 93.2 id, 0.6 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 7943.6 total, 2345.7 free, 3456.7 used, 2141.2 buff/cache

PID	USER	PR	NI	VIRT	RES	%CPU	%MEM	TIME+	COMMAND
2345	alice	20	0	234567	123456	2.5	3.1	1:23.45	firefox
1234	nginx	20	0	123456	23456	0.1	1.2	0:10.12	nginx
1	root	20	0	2256	1500	0.0	0.1	0:02.11	systemd

Keys to know:

q: quit P: sort by CPU M: sort by Memory 1: per-core view

HTOP - BETTER THAN TOP

Interactive and easy to use.

CPU[|||||]5.6%Tasks: 187, 1 running

Mem[|||||]1.26/7.7GLoad average: 0.45 0.32 0.28

Swp[|]0K/2.0GUptime: 2 days, 03:14:22

PID	USER	PRI	NI	VIRT	RES	%CPU	%MEM	TIME+	Command
2345	alice	20	0	234567	123456	2.5	3.1	1:23.45	firefox
1234	nginx	20	0	123456	23456	0.1	1.2	0:10.12	nginx
5678	root	20	0	34567	5678	0.0	0.7	0:05.21	containerd

F1Help

P2Setup

P3Search

F4Filter

P9Kill

F10Quit

PGREP - FIND PROCESS BY NAME

Searches for processes by name.

`$ pgrep nginx`
1234
5678



Options

- `pgrep -L nginx` # show PID and name
- `pgrep -u alice` # processes of user alice
- `pgrep -f "python app.py"` # match full command

KILL - TERMINATE A PROCESS

Stops a process using PID.

`$ kill 1234` # SIGTERM (graceful)
`$ kill -9 1234` # SIGKILL (force)

Common Signals

- 15 (SIGTERM) : Graceful shutdown
- 9 (SIGKILL) : Force kill
- 1 (SIGHUP) : Reload configuration
- 2 (SIGINT) : Interrupt (Ctrl+C)



KILLALL - KILL BY NAME

Kills all processes with a name.

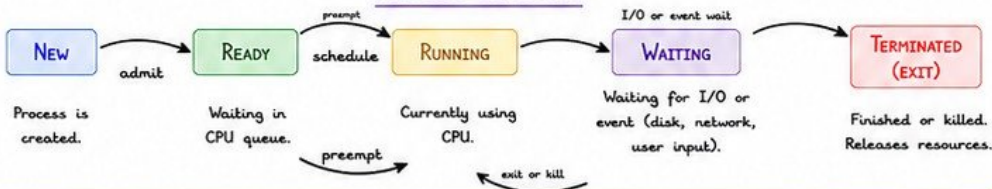
`$ killall nginx` # graceful
`$ killall -9 nginx` # force



Options

- `killall -u alice` # kill user's processes
- `killall -9 java` # exact match
- `pkill nginx` # same as killall nginx

PROCESS LIFECYCLE



STATES EXPLAINED

- R (Running) - Actively using CPU
- S (Sleeping) - Waiting / idle
- D (Uninterruptible) - Waiting for I/O
- T (Stopped) - Paused by signal
- Z (Zombie) - Finished, waiting for parent to read exit status

PRODUCTION EXAMPLES



- Find high CPU process: `top` → identify → kill or optimize
- Restart a crashed service: `pgrep service` → `kill` → `systemctl restart service`
- Memory leak troubleshooting: `ps aux --sort=-%mem`
- Limit runaway process: use `ulimit`, `cgroups`, or `systemd` limits



BEST PRACTICES

- ✓ Always try graceful kill (SIGTERM) first.
- ✓ Know what the process does before killing.
- ✓ Use `pgrep/ps` to confirm the correct PID.
- ✓ Monitor regularly with `top` or `htop`.
- ✓ Automate restarts with `systemd`.



COMMON MISTAKES

- Killing critical system processes.
- Using `kill -9` immediately.
- Ignoring zombie processes.
- Running heavy jobs as root.
- Not monitoring in production.



TIP: A zombie process (Z) has finished but the parent hasn't collected it.
Check with: `ps aux | grep Z`



REMEMBER:

You can't manage what you don't monitor.

VERIQTA

9. SYSTEM SERVICES

Services keep your system alive. systemd is the init system for most modern Linux distributions.

WHAT IS A SERVICE?

A service is a program that runs in the background to perform system tasks (web server, database, logging, monitoring, etc).

systemd is the init system and service manager for most Linux distributions.

SYSTEMD OVERVIEW

- PID 1 (init) process is systemd
- Manages services, sockets, targets, timers, mounts
- Starts early in boot and manages the system lifecycle
- Provides dependency management and parallel startup
- Uses unit files to define how services run



COMMON SERVICE UNIT TYPES

	.service	Regular service
	.socket	Socket activation
	.target	Group of units
	.mount	Mount points
	.timer	Timer based activation
	.path	Path based activation

SYSTEMCTL - THE PRIMARY COMMAND

`$ systemctl [command] [service]` # Example: `systemctl start nginx`



START A SERVICE

Starts a stopped service.

`$ sudo systemctl start nginx`



STOP A SERVICE

Stops a running service.

`$ sudo systemctl stop nginx`



RESTART A SERVICE

Stops and starts the service again.

`$ sudo systemctl restart nginx`



ENABLE A SERVICE

Starts on boot (creates required symlinks).

`$ sudo systemctl enable nginx`

Symlink created in `/etc/systemd/system/multi-user.target.wants/`



DISABLE A SERVICE

Does not start on boot.

`$ sudo systemctl disable nginx`

Symlinks removed from wants directory.



RELOAD A SERVICE

Reloads configuration without stopping. (If supported by the service)

`$ sudo systemctl reload nginx`

CHECK SERVICE STATUS

Shows current status and recent logs.

`$ sudo systemctl status nginx`

Look for:

- ✓ Active: active (running)
- ✓ Loaded: loaded (/usr/lib/systemd/system/nginx.service)
- ✓ Docs: man:nginx(8)
- ✓ Main PID
- ✓ Memory / CPU usage



LIST SERVICES

List all loaded services.

`$ systemctl list-units --type=service --state=running`

Or show all services:

`$ systemctl list-unit-files --type=service`

VIEW SERVICE LOGS

View logs for a service via journalctl.

`$ sudo journalctl -u nginx -f`

`-u nginx` # unit name
`-f` # follow logs
`-n 100` # last 100 lines



SERVICE TROUBLESHOOTING WORKFLOW



1. Check Status

`$ systemctl status <service>`
Is it active? Any errors?



2. Check Logs

`$ journalctl -u <service> -xe`
Look for error messages.



3. Verify Config

Check configuration files in `/etc/<service>/` or `/etc/<service>.conf`



4. Check Dependencies

`$ systemctl list-dependencies <service>`
Make sure required services are running.



5. Restart & Test

`$ systemctl restart <service>`
Test functionality.

USEFUL SYSTEMCTL COMMANDS

<code>\$ systemctl is-active nginx</code>	# check if service is running
<code>\$ systemctl is-enabled nginx</code>	# check if service starts on boot
<code>\$ systemctl daemon-reload</code>	# reload systemd after unit file changes
<code>\$ systemctl daemon-reexec</code>	# re-exec systemd
<code>\$ systemctl reset-failed</code>	# clear failed state
<code>\$ systemctl show nginx</code>	# show details about service



COMMON ISSUES & FIXES

- ✗ Service failed to start
→ Check logs: `journalctl -u <service> -xe`
- ✗ Permission denied
→ Check file permissions / user in unit file
- ✗ Port already in use
→ `netstat -tulnp | grep <port>`
- ✗ Dependency not met
→ `systemctl list-dependencies <service>`

Sample Unit File
`/etc/systemd/system/myapp.service`

[Unit]
Description=My App Service
After=network.target

[Service]
ExecStart=/usr/bin/myapp
User=appuser
Restart=on-failure

[Install]
WantedBy=multi-user.target

BEST PRACTICES

- ✓ Enable required services on boot.
- ✓ Use specific users (User=) in unit files.
- ✓ Restart services gracefully.
- ✓ Monitor services and logs regularly.



REMEMBER

systemctl is your best friend.
Know your services, their status, and their logs.



TIP

After editing a unit file:
1. `sudo systemctl daemon-reload`
2. `sudo systemctl restart <service>`

VERIQTA

10. LOGS

Logs tell the story
of your system.
Read them well,
solve problems fast.



WHAT ARE LOGS?

Logs are timestamped records of events, errors, warnings and activities generated by the OS, services and applications.

In production, logs are your best friend.

TYPES OF LOGS

- ✓ System logs (kernel, boot, hardware)
- ✓ Service logs (nginx, sshd, mysql, docker)
- ✓ Security logs (login attempts, sudo, auth failures)
- ✓ Application logs (apps, custom logs)
- ✓ Audit logs (who did what, when)

WHERE LOGS ARE STORED



/var/log

Most logs live in /var/log.

Some logs are managed by systemd (journald) and not stored as files.

ESSENTIAL LOG COMMANDS

JOURNALCTL - QUERY SYSTEMD LOGS



Query and filter logs from systemd (journald).

```
$ journalctl
$ journalctl -xe
$ journalctl -u nginx
$ journalctl -u sshd --since "1 hour ago"
$ journalctl --boot -1
```

Useful options:

- u <service> : show logs for a service
- xe : show recent logs with errors
- since : show logs from a time
- boot -1 : previous boot logs

VIEW LOG DIRECTORY



Navigate to log directory.

```
$ cd /var/log
$ ls -lah
```

Common subdirectories:

- /var/log/nginx
- /var/log/mysql
- /var/log/secure or /var/log/auth.log
- /var/log/syslog
- /var/log/dmesg

SYSLOG - SYSTEM LOG



General system messages (wide range of events).

```
$ cat /var/log/syslog
$ tail -f /var/log/syslog
```

Shows:

- Boot messages
- System services
- Network events
- Errors and warnings

AUTH.LOG / SECURE - AUTHENTICATION LOGS



Authentication and security events.

```
$ cat /var/log/auth.log
$ tail -f /var/log/auth.log
```

Shows:

- Successful logins
- Failed login attempts
- sudo usage
- SSH login events

DMSG - KERNEL RING BUFFER



Displays kernel messages (hardware, drivers, boot).

```
$ dmesg
$ dmesg -T | less
$ dmesg | tail -n 50
```

Useful for:

- Hardware issues
- Driver errors
- Boot problems
- Kernel panics

FOLLOW LOG IN REAL TIME



Live stream of log file updates.

```
$ tail -f /var/log/syslog
$ tail -f /var/log/nginx/error.log
$ journalctl -f -u nginx
```

Great for:

- Debugging in real time
- Monitoring errors
- Watching application activity

FILTER LOGS LIKE A PRO



Find what matters quickly.

```
$ grep -i error /var/log/syslog
$ grep -i failed /var/log/auth.log
$ journalctl -u sshd | grep -i fail
```

Useful patterns:

- error, fail, denied, timeout, refused
- ip address or user
- specific service name

CLEAR (ROTATE) LOGS



Logs grow! Rotate to save space.

```
$ sudo logrotate -f /etc/logrotate.conf
```

Logrotate:

- Compresses old logs
- Keeps size in control
- Prevents disk full issues

READING PRODUCTION LOGS - WORKFLOW



1. Identify the issue
What's the problem?
Slow app? Failed login?
Service down?



2. Find the right log
Check the relevant log:
 - Application
 - System
 - Security



3. Read recent logs
Use tail, journalctl or grep to find recent events.



4. Filter & analyze
Look for errors, warnings, patterns or timestamps.



5. Take action
Fix issue, restart service, or update configuration.



6. Verify & monitor
Confirm fix and monitor logs for new errors.



PRO TIP

Always check time!
Logs are useless if your server time is wrong.

TROUBLESHOOTING EXAMPLES

SCENARIO	WHERE TO LOOK	WHAT TO LOOK FOR	COMMAND EXAMPLE
Website 500 Error	/var/log/nginx/error.log Journal: nginx	Error messages, upstream failures, permission denied	\$ tail -f /var/log/nginx/error.log \$ journalctl -u nginx -xe
Can't SSH Login	/var/log/auth.log Journal: sshd	Failed password, invalid user, too many attempts (brute force)	\$ tail -f /var/log/auth.log \$ grep -i "failed" /var/log/auth.log
System Slow	/var/log/syslog Journal (systemd)	High load, OOM kill, disk full, service crashes	\$ journalctl -xe --since "30 min ago" \$ grep -i error /var/log/syslog
Service Not Starting	journalctl -u <service> /var/log/<service>/	Start failure, config error, missing dependency	\$ journalctl -u mysql -xe \$ tail -f /var/log/mysql/error.log
Disk Full	/var/log/syslog /var/log/messages	No space left on device, cleanup needed	\$ df -h \$ grep -i "no space" /var/log/syslog



BEST PRACTICES

- ✓ Know where your logs live.
- ✓ Monitor logs regularly.
- ✓ Use log rotation (logrotate).
- ✓ Centralize logs (ELK, Loki, CloudWatch).
- ✓ Set alerts for critical errors.



COMMON LOG MISTAKES

- ✗ Ignoring logs.
- ✗ Not checking the right log.
- ✗ Not syncing server time (use NTP).
- ✗ Not rotating logs (disk fills up).
- ✗ Deleting logs before investigation.



TIP: Logs don't lie. They show what really happened.
Read them like a detective, act like an engineer.



REMEMBER

When in doubt, check logs first.
Logs save hours of guesswork.



USEFUL TOOLS

journalctl, grep, awk, sed, tail, less, logrotate, rsyslog, logwatch, fail2ban.

VERIQTA

11. PACKAGE MANAGEMENT

Package management keeps your system secure, updated and reliable. Right package, right version, right time.



WHAT IS PACKAGE MANAGEMENT?



Package management tools automate the process of installing, upgrading, configuring and removing software on Linux systems. They also handle dependencies and updates.

WHY IT MATTERS IN PRODUCTION

- ✓ Keeps systems secure and patched
- ✓ Ensures version consistency
- ✓ Simplifies installation and rollback
- ✓ Reduces human error
- ✓ Supports automation and CI/CD



COMMON LINUX FAMILIES

- Debian/Ubuntu → apt / dpkg
- RHEL/CentOS/Alma/Rocky → dnf / yum / rpm
- SUSE → zypper (not covered here)

PACKAGE MANAGERS OVERVIEW

APT (ADVANCED PACKAGE TOOL)



Used in Debian, Ubuntu and derivatives.

```
$ sudo apt update
$ sudo apt install nginx
$ sudo apt upgrade
```

Features

- Simple & powerful
- Handles dependencies automatically
- Uses /etc/apt/sources.list

DNF (DANDY YUM)



Used in Fedora, RHEL 8+ AlmaLinux, Rocky Linux 8+.

```
$ sudo dnf update
$ sudo dnf install nginx
$ sudo dnf remove nginx
```

Features

- Fast and modern
- Better dependency resolution
- Uses /etc/yum.repos.d/*.repo

YUM (YELLOWDOG UPDATER MODIFIED)



Used in RHEL 7, CentOS 7 and older systems.

```
$ sudo yum update
$ sudo yum install nginx
$ sudo yum remove nginx
```

Features

- Legacy for older distros
- Similar to dnf
- Uses /etc/yum.repos.d/*.repo

RPM (REDHAT PACKAGE MANAGER)



Low-level package manager used by RHEL-based systems.

```
$ rpm -ivh package.rpm
$ rpm -Uvh package.rpm
$ rpm -e package
```

Features

- Installs single .rpm files
- No dependency resolution
- Used by dnf/yum under the hood

DPKG (DEBIAN PACKAGE)



Low-level package manager used by Debian-based systems.

```
$ dpkg -i package.deb
$ dpkg -r package
$ dpkg -l | grep nginx
```

Features

- Installs single .deb files
- No dependency resolution
- Used by apt under the hood

UPDATING SAFELY IN PRODUCTION



1 Check current status

```
$ apt list --upgradable # Debian/Ubuntu
$ dnf check-update      # RHEL/Fedora
$ yum check-update      # RHEL 7 / CentOS 7
```



2 Update package index (metadata)

```
$ sudo apt update
$ sudo dnf makecache
$ sudo yum makecache
```



3 Upgrade packages

```
$ sudo apt upgrade # safe upgrade (does not remove pkgs)
$ sudo apt full-upgrade # upgrades and may remove old pkgs
$ sudo dnf upgrade
$ sudo yum update
```



4 Reboot if required (check release notes)

```
$ sudo reboot
```



⚠ Tip: Always test updates in staging before production!

REPOSITORY MANAGEMENT



APT (Debian/Ubuntu)

- Repos configured in /etc/apt/sources.list or /etc/apt/sources.list.d/*.list
- Update after adding or changing repos.

```
# Add a PPA
$ sudo add-apt-repository ppa:nginx/stable
$ sudo apt update
```

```
# Example entries
$ deb http://archive.ubuntu.com/ubuntu jammy main
$ deb http://security.ubuntu.com/ubuntu jammy-security main
```

DNF/YUM (RHEL/Fedora)

- Repos in /etc/yum.repos.d/*.repo
- Each repo has a .repo file.

```
# Example: nginx.repo
[nginx-stable]
name=nginx stable repo
baseurl=https://nginx.org/packages/centos/7/$basearch/
gpgcheck=1
gpgkey=https://nginx.org/keys/nginx-signing.key
enabled=1
```

USEFUL REPO COMMANDS

```
# APT
$ sudo apt update
$ apt-cache search nginx
$ apt policy nginx

# DNF/YUM
$ sudo dnf repolist
$ sudo dnf repoinfo nginx-stable
$ sudo yum repolist all
```

REPO BEST PRACTICES

- ✓ Use official or trusted repositories
- ✓ Enable GPG key verification
- ✓ Pin versions for critical packages
- ✓ Mirror repos internally for reliability
- ✓ Remove unused/old repositories

COMMON PACKAGE TASKS CHEAT SHEET

TASK	APT (DEBIAN/UBUNTU)	DNF (RHEL 8+/FEDORA)	YUM (RHEL 7 / CENTOS 7)	RPM (LOW-LEVEL)	DPKG (LOW-LEVEL)
Update index	<code>sudo apt update</code>	<code>sudo dnf makecache</code>	<code>sudo yum makecache</code>	N/A	N/A
Search package	<code>apt search nginx</code>	<code>dnf search nginx</code>	<code>yum search nginx</code>	<code>rpm -qa grep nginx</code>	<code>dpkg -l grep nginx</code>
Install package	<code>sudo apt install nginx</code>	<code>sudo dnf install nginx</code>	<code>sudo yum install nginx</code>	<code>rpm -ivh package.rpm</code>	<code>dpkg -i package.deb</code>
Remove package	<code>sudo apt remove nginx</code>	<code>sudo dnf remove nginx</code>	<code>sudo yum remove nginx</code>	<code>rpm -e nginx</code>	<code>dpkg -r nginx</code>
Upgrade package	<code>sudo apt upgrade nginx</code>	<code>sudo dnf upgrade nginx</code>	<code>sudo yum update nginx</code>	<code>rpm -Uvh package.rpm</code>	<code>dpkg -i package.deb</code>
List installed	<code>dpkg -l</code>	<code>dnf list installed</code>	<code>yum list installed</code>	<code>rpm -qa</code>	<code>dpkg -l</code>



TROUBLESHOOTING COMMON ISSUES

- Package not found
→ Check repo enabled & updated
`$ sudo apt update` or `$ sudo dnf repolist`
- GPG key error
→ Import the repo key
`$ sudo rpm --import <key-url>`
- Dependency error
→ Use proper repo or resolve deps
`$ sudo dnf install --best --allowdowngrading pkg`
- Broken packages
→ Fix and clean cache
`$ sudo apt --fix-broken install`
`$ sudo dnf clean all && sudo dnf distro-sync`



PRO TIP

Document changes. Know what you installed, why and which version is running.



SECURITY REMINDER

Keep systems updated, but never at the expense of stability. Test first!



ROLLBACK TIP

Keep previous working versions available. Use version pinning when needed.

VERIQTA

12. DISK MANAGEMENT

Storage is the foundation of every system.
Monitor, manage and protect it always.



WHAT IS DISK MANAGEMENT?

Disk management includes monitoring usage, partitioning, mounting, unmounting and analyzing storage resources to ensure availability, performance and reliability.

WHY IT MATTERS IN PRODUCTION

- ✓ Prevent disk full outages
- ✓ Optimize storage usages
- ✓ Ensure proper mounts at boot
- ✓ Monitor growth and plan capacity
- ✓ Protect data and maintain performance



COMMON FILE SYSTEMS

- ext4 - Default, stable, robust
- xfs - High performance, scalable
- btrfs - Snapshots, checksums
- nfs - Network File System
- tmpfs - In-memory filesystem

ESSENTIAL DISK COMMANDS



DF - DISK FREE

Shows disk space usage of mounted filesystems.

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        50G   18G   28G   40% /
/dev/sdb1       100G   60G   40G   60% /data
tmpfs            2G     1M    2G    1% /run
```

Options

- h Human readable
- T Show filesystem type
- i Show inode usage



DU - DISK USAGE

Shows space used by files and directories.

```
$ du -sh /var/log
1.2G /var/log
$ du -ah /var | sort -hr | head -10
```

Options

- s Summary
- h Human readable
- a Show all files
- max-depth=1 Limit depth



LSBLK - LIST BLOCK DEVICES

Displays block devices and their relationships.

```
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
sda   8:0    0 100G 0 disk
├─sda1 8:1    0  1G 0 part /boot
├─sda2 8:2    0  50G 0 part /
├─sda3 8:3    0  49G 0 part /data
sdb   8:16   0 500G 0 disk
└─sdb1 8:17   0 500G 0 part /backup
```

Options

- f Show filesystem info
- o Customize output



FDISK - PARTITION TOOL

Create, delete and manage disk partitions.

```
$ sudo fdisk /dev/sdb
Command (m for help): p
Command (m for help): n
Command (m for help): d
Command (m for help): w
```

Common Commands

- p Print partition table
- n New partition
- d Delete partition
- w Write changes



MOUNT - ATTACH FILESYSTEM

Mounts a filesystem to a directory.

```
$ sudo mount /dev/sdb1 /data
```

Example (with options)

```
$ sudo mount -t ext4 /dev/sdb1 /data
$ sudo mount -o ro /dev/sdb1 /data
$ sudo mount -o noatime /dev/sdb1 /data
```



UMOUNT - DETACH FILESYSTEM

Unmounts a filesystem from a directory.

```
$ sudo umount /data
```

Force unmount (if busy)

```
$ sudo umount -l /data # lazy
$ sudo umount -f /data # force
```



BLKID - SHOW DEVICE INFO

Displays UUID, type and label of block devices.

```
$ sudo blkid
/dev/sda1: UUID="A1B2-3C4D"
TYPE="ext4" PARTUUID="01"
/dev/sdb1: UUID="F2E4-5D6C"
TYPE="xfs" PARTUUID="01"
```

Useful for:

- Identifying devices by UUID
- Using in /etc/fstab



SWAPON / SWAPOFF - MANAGE SWAP

Enable or disable swap space.

```
$ swapon -s
Filename      Type      Size  Used  Priority
/dev/sda3    partition  2G     0B    -2
```

```
$ sudo swapon /dev/sda3
$ sudo swapoff /dev/sda3
```



DISK USAGE ALERTING

Monitor disk usage and alert before it reaches critical levels.

```
$ df -h | awk 'NR==2{print $5}' | sed 's/%//'
```

Example alert (via cron/script)

```
#!/bin/bash
THRESHOLD=80
USAGE=$(df -h / | awk 'NR==2{print $5}' | sed 's/%//')
if [ $USAGE -ge $THRESHOLD ]; then
    echo "ALERT: Disk Almost Full - $USAGE% used!" | \
    mail -s "Disk Alert" admin@yourdomain.com
fi
```

PRODUCTION STORAGE MONITORING



FIND LARGE DIRECTORIES

Find directories consuming the most space.

```
$ sudo du -xh / | sort -hr | head -10
```



INODE USAGE (CHECK FILE COUNT)

High inode usage can cause "No space left on device".

```
$ df -i
Filesystem      Inodes   IUsed   IFree  IUse% Mounted on
/dev/sda2      262144  200000  62144    76% /
```



MONITOR DISK I/O

Check disk read/write performance.

```
$ iostat -xz 1 5
$ dstat -d 1 5
```

Key metrics:

- %util (> 80% is high)
- r/s, w/s (reads/writes per second)
- await (high = slow response)

PERSISTENT MOUNTS (AUTOMOUNT AT BOOT)

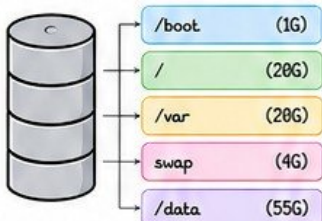
Use /etc/fstab to mount filesystems automatically.

#	<device>	<mount point>	<type>	<options>	<dump>	<pass>
1	UUID=A1B2-3C4D	/data	ext4	defaults,nofail	0	2
2	UUID=F2E4-5D6C	/backup	xfs	defaults	0	2

Best Practices

- ✓ Use UUID instead of device name
- ✓ Use defaults,nofail to avoid boot issues
- ✓ Test with: `sudo mount -a`

COMMON DISK LAYOUT (EXAMPLE)



COMMON DISK ISSUES & FIXES

- ✗ Disk full
→ Remove old logs, clean cache, extend disk
- ✗ High inode usage
→ Delete unnecessary files, reduce small files
- ✗ Filesystem read-only
→ Check with: `dmesg | tail`
→ Run `fck`, remount RW
- ✗ Slow disk I/O
→ Check `%util`, `await`, consider SSD or tuning



TIP: Monitor, plan and automate your storage operations. A full disk doesn't just slow down — it stops everything!

VERIQTA

13. MEMORY MANAGEMENT

Memory is precious.
Manage wisely,
monitor always!
Prevent problems
before they happen.



WHAT IS MEMORY MANAGEMENT?

Memory management ensures your system has enough RAM available for processes and the OS to run smoothly without slowdowns or out-of-memory (OOM) crashes.

WHY IT MATTERS IN PRODUCTION

- ✓ Prevents performance degradation
- ✓ Avoids OOM kills and service crashes
- ✓ Ensures stable applications
- ✓ Helps in capacity planning
- ✓ Improves user experience

LINUX MEMORY OVERVIEW



- Used (by processes)
- Cache / Buffers (reclaimable)
- Free (immediately available)
- Swap (disk-based)

ESSENTIAL MEMORY COMMANDS

FREE - MEMORY SUMMARY

`>_` Shows used, free, shared, buffer & cache memory.

```
$ free -h
total used free shared buff/cache available
Mem:  7.6Gi 2.1Gi 1.3Gi 230Mi  4.2Gi  4.9Gi
Swap:  2.8Gi   0B   2.8Gi
```

Options

- m Show in MB
- g Show in GB
- s <interval> Repeated update

VMSTAT - SYSTEM PERFORMANCE



Reports virtual memory statistics and system performance.

```
$ vmstat 1 5
procs -----memory----- --swap-- -----cpu-----
r b swpd free buff cache si so bi bo in cs us sy id wa st
0 0 0 135004 41200 276400 0 0 12 34 152 218 5 3 91 1 0
0 0 0 136104 41200 276506 0 0 32 146 265 4 2 98 1 0
1 0 0 136000 41240 276430 0 0 6 24 150 211 6 2 92 1 0
0 0 0 136200 41260 276880 0 0 8 36 155 215 4 3 92 1 0
```

Useful for:

- Identifying memory pressure
- Monitoring swap usage & I/O wait

SWAPPINESS - SWAP BEHAVIOR



Controls how aggressively the kernel uses swap.

```
$ cat /proc/sys/vm/swappiness
60
$ sudo sysctl -w vm.swappiness=10
```

Recommended values:

- 10 - Servers (prefer RAM)
- 60 - Default
- 100 - Swap used aggressively

BUFFERS & CACHE - PERFORMANCE BOOSTERS



Used memory that can be reclaimed when needed.

```
$ free -h
total used free shared buff/cache available
Mem:  7.6Gi 2.1Gi 1.3Gi 230Mi  4.9Gi
```

Tips

- Buffers - Metadata for block devices
- Cache - Recently used data (page cache)

LINUX MEMORY BREAKDOWN

Used (By Processes)	→ Memory actively used by running applications and system.
Cache (Reclaimable)	→ Disk cache used to speed up file access. Can be reclaimed when needed.
Buffers (Reclaimable)	→ Temporary storage for block device metadata. Also reclaimable.
Free (Immediately Available)	→ Memory not in use and available for applications.
Swap (On Disk)	→ Disk space used when RAM is full. Very slow compared to RAM.

★ Linux uses free memory for cache/buffers to improve performance. That's a GOOD thing!



OOM KILLER - OUT OF MEMORY PROTECTION

When memory is critically low, Linux may kill processes to protect the system.

```
$ dmesg | grep -i oom
[ 12345.678] Out of memory: Kill process 4321 (java)
score 987 or sacrifice child
[ 12345.679] Killed process 4321 (java) total-vm:204800kB,
anon-rss:150000kB, file-rss:1024kB, shmem-rss:0kB
```

Adjust OOM behavior

```
$ sudo sysctl -w vm.panic_on_oom=0 # Don't panic system
$ sudo sysctl -w vm.oom_kill_allocating_task=1 # Kill allocating task
```

Check(Per-process) OOM score

```
$ cat /proc/<pid>/oom_score # 0 - 1000 (higher = more likely killed)
$ cat /proc/<pid>/oom_score_adj # Adjust priority (-1000 to 1000)
```

UNDERSTANDING MEMORY METRICS

Metric	Meaning
total	Total physical memory
used	Memory used by processes
free	Memory immediately free
shared	Memory shared between processes
buff/cache	Buffers + Cache (reclaimable)
available	Memory available for new apps
swap used	Swap currently in use
swap free	Swap available



"available" is the best number to check for real free memory!

MEMORY TROUBLESHOOTING WORKFLOW



1. Detect

Check alerts
(High memory usage)



2. Analyze

Use commands:
`free -h`
`vmstat 1`
`ps aux --sort=-%mem`



3. Identify Cause

Find memory-hungry processes, leaks, or cache issues



4. Take Action

Optimize app, restart service, tune swappiness, or add more RAM



5. Monitor

Keep monitoring and set alerts to prevent recurrence

COMMON SCENARIOS



High Memory Usage

- Check top memory processes
- Look for memory leaks
- Restart or optimize



Swap Usage High

- System will be slow
- Reduce memory usage
- Add more RAM



OOM Kills in Logs

- Check dmesg for OOM events
- Adjust application memory
- Increase RAM / add swap



BEST PRACTICES

- ✓ Monitor memory usage 24/7
- ✓ Set alerts for high usage & swap
- ✓ Keep swappiness low on servers (10)
- ✓ Understand buffers & cache (they help!)
- ✓ Optimize applications & queries
- ✓ Plan capacity with future growth in mind



USEFUL COMMANDS CHEAT SHEET

```
$ free -h # Memory summary
$ vmstat 1 # Real-time memory stats
$ cat /proc/meminfo # Detailed memory info
$ ps aux --sort=-%mem # Top memory-consuming processes
$ top # Interactive process monitor
$ cat /proc/sys/vm/swappiness # Check swappiness
$ dmesg | grep -i oom # Check OOM killer logs
```

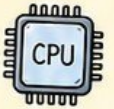


TIP: Memory is like oxygen — you don't miss it until it's gone! Monitor, understand, and act before users feel the pain.

VERIQTA

14. CPU MONITORING

The CPU is the engine
of your system.
Monitor it, understand it,
optimize it.



WHAT IS CPU MONITORING?

CPU monitoring helps you track processor usage, system load and performance trends to ensure applications remain responsive and systems stay healthy.

WHY IT MATTERS IN PRODUCTION

- ✓ Detect high CPU usage early
- ✓ Identify performance bottlenecks
- ✓ Prevent slow applications & timeouts
- ✓ Capacity planning and scaling
- ✓ Ensure a great user experience

CPU USAGE BREAKDOWN (top)

- User (apps, processes)
- System (kernel, daemons)
- I/O Wait (waiting for I/O)
- Idle (free CPU)
- Steal (in VM environments)



ESSENTIAL CPU COMMANDS

UPTIME - SYSTEM UP TIME



Shows how long the system has been up and the load average.

```
$ uptime
15:42:10 up 23 days,
2:15, 3 users,
load average: 0.42, 0.35, 0.30
```

Load average (1, 5, 15 min) shows average runnable or uninterruptible processes.

LOAD AVERAGE - SYSTEM LOAD



Indicates system load and how busy the system is over time.

```
$ cat /proc/loadavg
0.42 0.35 0.30 2/987 12345
```

Tips:

- ~1.0 per CPU core is normal
- > 1.0 consistently = high load
- Check what is causing the load

MPSTAT - CPU PERFORMANCE



Shows CPU usage per CPU/core in real time.

```
$ mpstat -P ALL 1 3
CPU %usr %sys %iowait %idle
all 12.51 5.23 1.25 81.01
0 11.90 4.90 1.10 82.10
1 13.10 5.40 1.40 80.10
```

Options:

- P ALL Show all CPUs/cores
- 1 Interval (sec)
- 3 Count (times)

SAR - SYSTEM ACTIVITY REPORTER



Collects, reports and saves system activity.

```
$ sar -u 1 3
12:00:01 %user %nice %kystem
12:00:02 11.20 0.00 4.00
12:00:03 10.90 0.00 5.10
Average: 11.05 0.00 4.95
```

Useful reports:

- sar -u CPU usage
- sar -q Run queue & load
- sar -r Memory
- sar -d Disk I/O

NICE - START WITH PRIORITY



Start a process with adjusted CPU priority.

```
$ nice -n 10 ./backup.sh
```

Lower nice (e.g. -n -10) = higher priority
Higher nice (e.g. -n 10) = lower priority

RENICE - CHANGE PRIORITY



Change priority of a running process.

```
$ renice -n 15 -p 12345
```

-n 15 New nice value
-p 12345 Process ID
Range: -20 (highest) to 19 (lowest)



UNDERSTANDING LOAD AVERAGE

Formula:

$$\text{Load Average} = \frac{\text{Running Processes} + \text{Uninterruptible (I/O waiting) Processes}}{\text{Number of CPU Cores}}$$

GOOD

- Load < or = number of CPU cores
- System is responsive
- Short spikes are normal

BAD

- Load >> number of CPU cores
- Sustained high load
- Users experience slow apps
- Check and investigate

QUICK CHECK EXAMPLE

System: 4 CPU cores

Load average: 8.0, 6.5, 5.0

- 8.0 > 4 cores → High load!
- System is under heavy stress
- Investigate top processes

Always correlate load with CPU utilization using tools like top, mpstat, sar



CPU UTILIZATION BREAKDOWN (top)

```
$ top -bn1 | head -n 7
```

```
%Cpu(s): 12.5 us, 5.2 sy, 1.2 ni, 2.1 id, 78.3 wa, 0.1 hi, 0.6 si, 0.0 st
```

User (apps) System (kernel) Nice (low prio) Idle (free) I/O Wait (waiting) Hardware Interrupt Soft Interrupt (VM)

- High %wa = Disk or I/O bottleneck
- High %sy = Kernel or system overhead
- High %us = Applications consuming CPU
- Low %id = CPU is busy (check why)

COMMON CPU BOTTLENECKS



High CPU Usage (%us)

- CPU-bound applications
- Inefficient code / loops
- Too many processes
- Crypto / compression tasks

Check: top, ps, pidstat

Action: Optimize app, scale out, or upgrade CPU



High System Time (%sy)

- Kernel overhead
- Too many context switches
- Drivers / system calls
- Network interrupts

Check: mpstat, vmstat

Action: Tune kernel, update drivers, optimize configs



High I/O Wait (%wa)

- Slow disks (HDD/Network)
- DB or FS I/O heavy ops
- Storage contention
- NFS/remote latency

Check: iostat, sar -d, top

Action: Improve storage, add cache, optimize queries



Run Queue / High Load

- More tasks than CPU cores
- CPU saturation
- Contention for resources

Check: uptime, top, mpstat

Action: Kill/optimize tasks, scale up/out, tune app



Too Many Interrupts

- High network interrupts
- Misconfigured hardware
- Driver issues

Check: cat /proc/interrupts

Action: Update drivers, adjust NIC settings

CPU MONITORING WORKFLOW



1. Detect
High load alert
(User report / Monitoring)

2. Diagnose
Check uptime,
top, mpstat,
sar

3. Identify Cause
Find CPU, I/O,
apps or kernel
issue

4. Take Action
Optimize, kill,
renice, scale
or tune

5. Monitor
Recheck &
validate
improvement

BEST PRACTICES

- ✓ Monitor CPU 24/7
- ✓ Set alerts on high CPU & load
- ✓ Track trends, not just spikes
- ✓ Correlate with memory, disk & network
- ✓ Automate remediation where possible

QUICK COMMAND CHEAT SHEET



```
uptime          # Load average & uptime
cat /proc/loadavg # Load average (1,5,15)
top / htop      # Real-time process view
mpstat -P ALL 1 3 # Per-CPU usage
sar -u 1 3       # Historical CPU stats
nice -n 10 cmd   # Start with low priority
renice -n 15 -p PID # Change priority of process
```



PRO TIP: A busy CPU doesn't always mean a problem. Understand the cause, not just the number!



REMEMBER: High load is a symptom, not the disease.

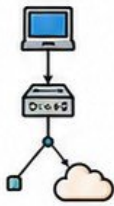


WARNING: Ignoring CPU issues can lead to slow apps, timeouts and unhappy users!

VERIQTA

15. NETWORKING BASICS

Networks connect
your systems.
Understand, test
and troubleshoot them
like a pro!



WHAT IS NETWORKING?

Networking allows systems to communicate using IP addresses, ports and protocols. Linux provides powerful tools to inspect and debug network connectivity.

WHY IT MATTERS IN PRODUCTION

- ✓ Ensure service connectivity and availability
- ✓ Debug slow applications and timeouts
- ✓ Verify firewall, routing and DNS issues
- ✓ Monitor ports, connections and traffic
- ✓ Improve security and reduce downtime

COMMON NETWORK CONCEPTS

- IP Address - Unique address of a device
- Subnet & CIDR - Network range
- Gateway - Exit point to other networks
- DNS - Translates domain to IP
- Port - Service access point (1-65535)
- Protocol - Rules: TCP / UDP / ICMP



ESSENTIAL NETWORK COMMANDS

IP - SHOW / CONFIGURE

Show or configure network interfaces and routes.

```
$ ip addr
$ ip a
$ ip route
```

Useful options:

- ip addr show eth0
- ip route show
- ip link set eth0 up

PING - TEST CONNECTIVITY

Test reachability using ICMP echo requests.

```
$ ping google.com
$ ping -c 4 8.8.8.8
$ ping -I eth0 1.1.1.1
```

Options:

- -c count
- -i interval
- -W timeout

SS - SOCKET STATISTICS

Modern tool to display sockets and connections.

```
$ ss -tulpen
$ ss -tn sport = :80
$ ss -a state established
```

Shows:

- Listening ports (-l)
- Established connections (-a)
- Process using ports (-p)

NETSTAT - LEGACY TOOL

Legacy tool (net-tools package) to show network info.

```
$ netstat -tulpen
$ netstat -tn
$ netstat -rn
```

Shows:

- -t : TCP
- -u : UDP
- -r : Routing table

TRACEROUTE - PATH DISCOVERY

Trace the route packets take to reach a host.

```
$ traceroute google.com
$ traceroute -n 1.1.1.1
$ traceroute -I google.com
```

Options:

- -n no DNS lookup
- -I ICMP mode
- -T : TCP mode

CURL - TRANSFER DATA

Transfer data from or to a server.

```
curl://
$ curl https://example.com
$ curl -I example.com
$ curl -v https://api.com
```

Uses:

- Test HTTP/HTTPS
- Check headers (-I)
- Debug with verbose (-v)

WGET - DOWNLOAD FILES

Non-interactive downloader great for scripts.

```
$ wget https://example.com/file.zip
$ wget -c https://example.com/file.zip
$ wget -O file.txt https://test.com/a.txt
```

Options:

- -c continue download
- -O save as
- -q quiet mode



COMMON PORTS REFERENCE

Port	Protocol	Service	Description
22	TCP	SSH	Secure Shell
53	TCP/UDP	DNS	Domain Name System
80	TCP	HTTP	Web (Unencrypted)
443	TCP	HTTPS	Web (Encrypted)
3306	TCP	MySQL	MySQL Database
5432	TCP	PostgreSQL	PostgreSQL Database
6379	TCP	Redis	Redis Cache
8080	TCP	HTTP-Alt	Alternative HTTP



CHECK OPEN PORTS QUICKLY

```
$ ss -tulpen
```

- LISTEN → Port is open and waiting
- ESTAB → Connection established
- TIME-WAIT → Connection closed recently
- CLOSE-WAIT → Unclosed by application



FIREWALL STATUS CHECK

```
$ sudo ufw status verbose
```

or

```
$ sudo firewall-cmd --list-all
```

Ensure only required ports are open!

DNS TROUBLESHOOTING GUIDE

1. CHECK DNS RESOLUTION

```
$ nslookup google.com
$ dig google.com
```

Should return IP address(es).

2. CHECK DNS SERVER

```
$ cat /etc/resolv.conf
```

Look for nameserver entries.

3. TEST DNS FROM SERVER

```
$ dig @8.8.8.8 google.com
```

Test with public DNS (8.8.8.8).

4. COMMON DNS ISSUES

- Wrong /etc/resolv.conf
- DNS server down
- Network unreachable
- Firewall blocking port 53

5. QUICK FIXES

- ✓ Correct resolv.conf
- ✓ Use working DNS
- ✓ Check firewall rules
- ✓ Restart network service

NETWORK INTERFACE INFO

```
$ ip addr show
```

Example output (shortened):

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
inet 192.168.1.10/24 brd 192.168.1.255 scope global eth0
valid_lft forever preferred_lft forever
inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
valid_lft forever preferred_lft forever
```

← IPv4 Address
← IPv6 Address

ROUTING TABLE

```
$ ip route show
```

Example output (shortened):

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
10.0.0.0/24 via 192.168.1.254 dev eth0
```

- default → Default gateway
- 192.168.1.0/24 → Local network
- 10.0.0.0/24 → Route to another network

NETWORK TROUBLESHOOTING FLOW

- 1 Can you ping the gateway?
- 2 Can you reach external IP (8.8.8.8)?
- 3 Can you resolve domain (nslookup)?
- 4 Can you connect to the service/port?
- 5 Check firewall / security / routes

Isolate the layer that fails!

PRO TIP?

Always verify from the OS level first before blaming applications or cloud providers.



REMEMBER

Network issues cause most outages, good visibility = fast resolution!

QUICK COMMAND RECAP

- | | | | |
|------------|---------------------|-----------------|--------------|
| ip addr | # Show IPs | traceroute host | # Trace path |
| ping host | # Test connectivity | curl -I url | # Check HTTP |
| ss -tulpen | # Show sockets | dig domain | # DNS lookup |

VERIQTA

16. SSH

Secure access
is not optional.
Lock it down,
sleep better.



WHAT IS SSH?



SSH (Secure Shell) provides secure remote login and command execution over an encrypted connection. It protects data and credentials from attackers.

WHY IT MATTERS IN PRODUCTION

- ✓ Encrypts all traffic (passwords, commands, data)
- ✓ Prevents eavesdropping and MITM attacks
- ✓ Allows secure file transfer
- ✓ Supports key-based authentication
- ✓ Essential for automation and remote management

SSH CONNECTION OVERVIEW



ESSENTIAL SSH COMMANDS

SSH - CONNECT

```
$ ssh user@server_ip
```

Examples:

- `$ ssh ec2-user@3.10.20.30`
- `$ ssh -p 2222 admin@host`

Options:

- p Specify port
- i Use identity file (key)
- o Override options

SSH KEYS - GENERATE

```
$ ssh-keygen -t ed25519
```

Examples:

- `$ ssh-keygen -t rsa -b 4096`
- `$ ssh-keygen -C "mykey"`

Key Files:

- `~/.ssh/id_ed25519` (private)
- `~/.ssh/id_ed25519.pub` (public)

SSH-COPY-ID - PUSH KEY

```
$ ssh-copy-id user@host
```

Example:

- `$ ssh-copy-id ec2-user@3.10.20.30`

What it does:

Copies your public key to the remote server's `~/.ssh/authorized_keys` file.

SCP - SECURE COPY

```
$ scp file user@host:/path
```

Examples:

- `$ scp app.log ec2-user@host:/tmp`
- `$ scp -r mydir/ user@host:/backup`

Options:

- r Copy directories
- P Specify port
- i Use identity file

SFTP - SECURE FILE TRANSFER

```
$ sftp user@host
```

Inside sftp shell:

- `sftp> ls`
- `sftp> put file.txt`
- `sftp> get file.txt`
- `sftp> cd /var/log`
- `sftp> bye`

SSH CONFIG FILE

```
$ nano ~/.ssh/config
```

Example:

```
Host prod
  HostName 3.10.20.30
  User ec2-user
  Port 22
  IdentityFile ~/.ssh/id_ed25519
```

Then connect with:

```
$ ssh prod
```

SECURE ACCESS BEST PRACTICES

- ✓ Always use key-based authentication
- ✓ Disable password login
- ✓ Use non-standard SSH port
- ✓ Limit access with firewall
- ✓ Keep SSH server updated
- ✓ Monitor and log SSH access
- ✓ Use bastion host for internal access



SSH HARDENING CHECKLIST

- 1 Change SSH port from 22
- 2 Disable root login
- 3 Disable password authentication
- 4 Allow only specific users
- 5 Use key-based authentication only
- 6 Set idle timeout (ClientAliveInterval)
- 7 Limit concurrent sessions (MaxSessions)
- 8 Fail2ban: block brute-force attempts



EXAMPLE: /ETC/SSH/SSHD_CONFIG (HARDENED)

```
Port 2222
Protocol 2
PermitRootLogin no
PasswordAuthentication no
ChallengeResponseAuthentication no
UsePAM yes
AllowUsers ec2-user deployer
MaxAuthTries 3
MaxSessions 2
ClientAliveInterval 300
ClientAliveCountMax 2
```

After changes:

1. Restart SSH service
2. Test connection
3. Update firewall

```
$ sudo systemctl restart sshd
```

SSH WORKFLOW

1. GENERATE KEY



```
$ ssh-keygen -t ed25519
```

Creates private & public key pair.

2. COPY PUBLIC KEY



```
$ ssh-copy-id user@host
```

Adds key to server's `authorized_keys`.

3. CONNECT SECURELY



```
$ ssh user@host
```

Login without password using your key.

4. TRANSFER FILES



```
$ scp file user@host:/path
```

or

```
$ sftp user@host
```


Securely copy or manage files.

5. REPEAT SECURELY



Enjoy secure, encrypted remote access!



TROUBLESHOOTING SSH

- ✗ Connection refused
→ Check SSH service: `sudo systemctl status sshd`
- ✗ Permission denied (publickey)
→ Check key permissions:
`chmod 700 ~/.ssh`
`chmod 600 ~/.ssh/id_*`
- ✗ Too many authentication failures
→ Check logs: `sudo journalctl -u sshd`
→ Use fail2ban to block attackers
- ✗ Can't connect on port
→ Check firewall: `sudo ufw status`

COMMON SSH PORTS

Port	Service	Use
22	SSH	Default SSH port
2222	SSH	Custom (recommended)
2200	SSH	Alternative SSH
443	SSH over HTTPS	
8022	SSH	Another alternative



Using a non-standard port reduces automated attack noise.



QUICK COMMAND CHEAT SHEET

```
$ ssh user@host # Connect
$ ssh -i key.pem user@host # Connect with key
$ ssh user@host -p 2222 # Connect on custom port
$ ssh-copy-id user@host # Copy public key
$ scp file user@host:/path # Copy file to server
$ scp user@host:/path file # Copy file from server
$ sftp user@host # Start SFTP session
$ exit / logout # Close SSH session
```



PRO TIP: Treat SSH access like the keys to your kingdom. Protect it, monitor it, and never share it.



REMEMBER: Security isn't a feature, it's a mindset!

VERIQTA

17. BASH SCRIPTING BASICS

Automate the boring.
Eliminate human error.
Save time.
Build reliable systems.



WHAT IS BASH SCRIPTING?

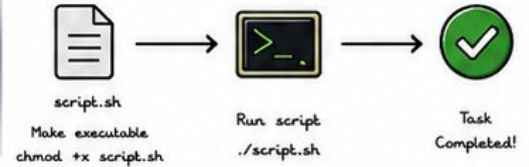
```
#!/bin/bash
```

Bash scripting lets you automate tasks, manage systems, and solve problems using simple and powerful shell scripts.

WHY IT MATTERS IN PRODUCTION

- ✓ Automate repetitive tasks
- ✓ Ensure consistency and reliability
- ✓ Speed up deployments and operations
- ✓ Reduce human errors
- ✓ Improve productivity and monitoring

BASH SCRIPT EXECUTION



CORE BASH CONCEPTS

1. VARIABLES

No types, no declaration.

```
name="VERIQTA"
count=10
path="/var/log"
```

Use:

- \$name
- \${name}
- \${name:-default}



2. LOOPS

For Loop

```
for i in 1 2 3 4 5; do
  echo "Number: $i"
done
```

While Loop

```
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  ((count++))
done
```



3. CONDITIONS

If Statement

```
if [ -f /etc/passwd ]; then
  echo "File exists"
else
  echo "File not found"
fi
```

Common Operators

```
-eq Equal
-ne Not equal
-gt Greater than
-lt Less than
-z String is empty
-f File exists
-d Directory exists
```



4. FUNCTIONS

Create reusable blocks of code.

```
my-function() {
  echo "Hello $1"
  echo "Today is $(date)"
}
```

```
# Call function
my-function "VERIQTA"
```

QUICK TIPS

- ★ Always use `#!/bin/bash` at the top
- ★ Use `set -e` to exit on error
- ★ Quote variables: `"$var"`
- ★ Use meaningful names
- ★ Comment your code



AUTOMATION EXAMPLES



1. CHECK DISK USAGE ALERT

Alert if disk usage > 80%

```
#!/bin/bash
THRESHOLD=80
USAGE=$(df / | awk 'NR==2{print $5}' | tr -d '%')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
  echo "WARNING: Disk usage is $(USAGE)% on $(hostname)" | mail -s "Disk Alert" admin@domain.com
fi
```



2. SERVICE HEALTH CHECK

Restart service if not running

```
SERVICE=nginx
if ! systemctl is-active --quiet $SERVICE; then
  echo "$SERVICE is down. Restarting..."
  systemctl restart $SERVICE
  echo "$SERVICE restarted at $(date)"
fi
```



3. BACKUP SCRIPT

Simple backup with timestamp

```
SRC="/var/www"
DEST="/backup"
DATE=$(date +%F_%H%M%S)
tar -czf "$DEST/backup_$DATE.tar.gz" "$SRC"
echo "Backup completed: backup_$DATE.tar.gz"
```



4. LOG CLEANUP

Delete logs older than 7 days

```
LOG_DIR="/var/log"
find $LOG_DIR -type f -name "*.log" -mtime +7 -exec rm -f {} \;
echo "Old logs removed."
```

PRODUCTION READY SCRIPTS

1. SYSTEM INFO REPORT

Collect and save system info

```
#!/bin/bash
OUT="/tmp/sys_report_$(date +%F).txt"
{
  echo "=== System Report ==="
  echo "Date: $(date)"
  echo "Hostname: $(hostname)"
  echo "Uptime: $(uptime -p)"
  echo "Disk Usage:"
  df -h
  echo "Memory Usage:"
  free -h
} > "$OUT"
echo "Report saved to $OUT"
```



2. BULK USER CREATION

Create users from a file

```
#!/bin/bash
FILE="users.txt"
while read -r user pass; do
  useradd -m "$user"
  echo "$user:$pass" | chpasswd
  echo "Created user: $user"
done < "$FILE"
```

3. SERVICE MONITOR

Monitor multiple services

```
#!/bin/bash
SERVICES=("nginx" "docker" "sshd")
for svc in "${SERVICES[@]}; do
  if systemctl is-active --quiet $svc; then
    echo "✓ $svc is running"
  else
    echo "✗ $svc is NOT running"
  fi
done
```

4. AUTO UPDATE SCRIPT

Update system safely

```
#!/bin/bash
echo "Updating system..."
apt update && apt upgrade -y
if [ $? -eq 0 ]; then
  echo "✓ Update successful"
  systemctl --no-pager --failed
else
  echo "✗ Update failed"
fi
```

SCRIPT TEMPLATE

```
#!/bin/bash
# Script: script_name.sh
# Description: What this script does
# Author: VERIQTA
# Date: $(date +%F)

set -e # Exit on error
# --- Your code here ---

echo "Script completed successfully."
```



BEST PRACTICES

- ✓ Start with `#!/bin/bash`
- ✓ Use `set -e` to pipefail
- ✓ Check errors and handle them
- ✓ Log important actions
- ✓ Test scripts before production
- ✓ Keep scripts simple and modular
- ✓ Use meaningful variable names

COMMON MISTAKES

- ✗ Forgetting `#!/bin/bash`
- ✗ Not quoting variables
- ✗ Ignoring errors
- ✗ Hardcoding sensitive information
- ✗ Not testing scripts
- ✗ Running scripts with root without understanding



BASH COMMAND CHEAT SHEET

```
echo # Print text
read # Read user input
pwd # Print working directory
cd # Change directory
ls # List files
cp # Copy files
mv # Move/rename files
rm # Remove files
cat # View file content
grep # Search text
chmod # Change permissions
chown # Change ownership
```



PRO TIP: Great scripts are simple, tested, and documented. Your future self will thank you!



Test in dev.
Run in prod.
Trust, but verify.



Automate today.
Dominate tomorrow!



VERIQTA

18. LINUX TROUBLESHOOTING

Troubleshooting is not guessing.
It's a systematic way to find the root cause and fix it right.



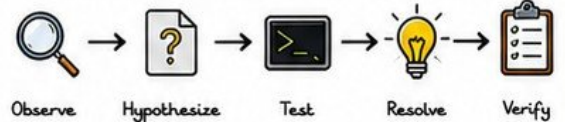
WHAT IS TROUBLESHOOTING?

Troubleshooting is the process of identifying, isolating and resolving issues in systems to restore normal operation with minimal impact.

WHY IT MATTERS IN PRODUCTION

- ✓ Minimize downtime and business impact
- ✓ Restore services quickly and safely
- ✓ Prevent repeat incidents
- ✓ Improve reliability and user trust
- ✓ Build confidence in critical environments

TROUBLESHOOTING MINDSET



COMMON PRODUCTION ISSUES & QUICK CHECKS

1. HIGH CPU



Symptoms:

- System slow
- High load average
- Processes not responding

Quick Checks:

```
$ top
$ htop
$ ps aux --sort=-%cpu | head
$ uptime
```

What to look for:

Processes using high CPU, runaway loops, forks, cron jobs.

2. HIGH MEMORY



Symptoms:

- System slow
- Swap usage increasing
- OOM kills in logs

Quick Checks:

```
$ free -h
$ top
$ vmstat 1 5
$ dmesg | grep -i oom
```

What to look for:

High memory usage, swap usage, OOM killer.

3. DISK FULL



Symptoms:

- Can't create files
- Services fail
- "No space left" errors

Quick Checks:

```
$ df -h
$ du -sh /* 2>/dev/null | sort -h
$ df -i
$ ls -lh /var/log
```

What to look for:

Partition at 100%, large files, log files, inode usage.

4. SERVICE DOWN



Symptoms:

- Service not running
- Application errors
- Connection refused

Quick Checks:

```
$ systemctl status <service>
$ systemctl list-units --failed
$ journalctl -u <service> -n 50
$ ss -tulpen | grep <port>
```

What to look for:

Service status, recent logs, port not listening.

5. NETWORK ISSUES



Symptoms:

- Can't reach hosts
- Slow connections
- DNS failures

Quick Checks:

```
$ ping <host>
$ ss -tulpen
$ traceroute <host>
$ dig <domain>
$ ip route
```

What to look for:

Routing, DNS, firewall, packet loss, latency.

6. GENERAL ISSUES



Symptoms:

- Unexpected behavior
- Errors in logs
- Application failures

Quick Checks:

```
$ journalctl -xe
$ tail -f /var/log/syslog
$ dmesg | tail
$ netstat -tulpen
```

What to look for:

Error messages, kernel issues, misconfigurations.

STEP-BY-STEP DEBUGGING WORKFLOW

1. IDENTIFY



- What is the problem?
- When did it start?
- Who is affected?
- What changed?

2. GATHER INFO



- Check metrics
- Check logs
- Check system state
- Reproduce if possible

3. ANALYZE



- Look for patterns
- Compare baseline
- Narrow down scope
- Form a hypothesis

4. TEST HYPOTHESIS



- Run commands
- Check configs
- Validate theory
- Confirm root cause

5. RESOLVE



- Apply fix
- Restart if needed
- Validate resolution
- Monitor closely

6. VERIFY & DOCUMENT



- Confirm service healthy
- Document root cause
- Document fix
- Prevent recurrence

USEFUL COMMANDS CHEAT SHEET

System Info	<code>uname -a</code> <code>uptime</code> <code>hostnamectl</code> <code>lscpu</code> <code>free -h</code>
Process	<code>top</code> <code>htop</code> <code>ps aux</code> <code>pgrep <name></code> <code>kill -9 <pid></code>
Memory	<code>free -h</code> <code>vmstat 1 5</code> <code>cat /proc/meminfo</code>
Disk	<code>df -h</code> <code>du -sh *</code> <code>lsblk</code> <code>fdisk -l</code>
Logs	<code>journalctl -xe</code> <code>journalctl -u <svc></code> <code>tail -f /var/log/syslog</code>
Services	<code>systemctl status <svc></code> <code>systemctl restart <svc></code> <code>systemctl --failed</code>
Network	<code>ip a</code> <code>ip r</code> <code>ss -tulpen</code> <code>ping <host></code> <code>traceroute <host></code>
DNS	<code>dig <domain></code> <code>nslookup <domain></code> <code>host <domain></code>

QUICK TRIAGE CHECKLIST

- ✓ Is the system reachable (SSH)?
- ✓ Check load average: `uptime`
- ✓ Check CPU usage: `top`
- ✓ Check Memory usage: `free -h`
- ✓ Check Disk usage: `df -h`
- ✓ Check Running services: `systemctl --failed`
- ✓ Check Recent logs: `journalctl -xe`
- ✓ Check Network connectivity: `ping`, `ss`, `ip r`
- ✓ Check DNS resolution: `dig`, `nslookup`
- ✓ Reproduce the issue (if possible)
- ✓ Escalate if needed with details

Always collect:

- ✗ Timestamp
- ✗ What you saw
- ✗ What you ran
- ✗ Output/Errors
- ✗ Steps taken
- ✗ Impact

This saves time for everyone!



PRO TIPS

- Check the basics first (CPU, Memory, Disk, Network).
- Read logs - they always tell a story.
- Don't guess - verify with data.
- Fix the root cause, not just the symptom.



REMEMBER

Good engineers fix problems.
Great engineers prevent them.

Observe → Understand → Fix → Improve



COMMON PITFALLS

- ✗ Restarting without understanding the issue
- ✗ Ignoring logs and alerts
- ✗ Fixing symptoms, not the cause
- ✗ No documentation or follow-up



VERIQTA

19. COMMON LINUX PRODUCTION MISTAKES

Mistakes happen.
Learning prevents
outages.
Document. Share.
Never repeat.



WHY MISTAKES HAPPEN

- Rushing changes
- Lack of understanding
- No checks or validations
- Poor documentation
- Overconfidence

Small mistake
=
Big impact

THE PRODUCTION IMPACT



Downtime



Lost users



Financial loss



Trust damage



Reputation hit

MINDSET FOR PRODUCTION

- ✓ Think before you type
- ✓ Verify before you execute
- ✓ Test in safe environment
- ✓ Document every change
- ✓ Learn and improve always



TOP 6 COMMON PRODUCTION MISTAKES

1. RUNNING AS ROOT



Using root for everyday tasks increases the blast radius of mistakes and security risks.

Example:
`rm -rf /` / executed as root 🚨

- Prevention:**
- ✓ Use regular user
 - ✓ Use sudo only when required
 - ✓ Enable sudo logging

2. WRONG PERMISSIONS



Incorrect permissions can cause outages or expose sensitive data.

Example:
`chmod 777 /etc`
Opening doors to attackers!

- Prevention:**
- ✓ Follow least privilege
 - ✓ Use ACLs carefully
 - ✓ Regularly audit permissions

3. DELETING CRITICAL FILES



One wrong command can remove critical system files.

Example:
`rm -rf /var/log/*`
`rm -rf /` (disaster)

- Prevention:**
- ✓ Double-check commands
 - ✓ Use `-i` (interactive)
 - ✓ Backup before changes

4. IGNORING LOGS



Ignoring logs hides problems until users are affected.

Example:
Errors in logs but nobody checked.

- Prevention:**
- ✓ Monitor and review logs
 - ✓ Set alerts
 - ✓ Act on warnings early

5. POOR BACKUPS



No backup or untested backup is a recipe for disaster.

Example:
Backup exists but restore never tested.

- Prevention:**
- ✓ Automate backups
 - ✓ Verify restore regularly
 - ✓ Keep offsite copies

6. NO CHANGE CONTROL



Making unplanned changes without testing or rollback plan is risky.

Example:
Random change during peak hours 🚨

- Prevention:**
- ✓ Use change process
 - ✓ Test in staging
 - ✓ Have rollback plan

PRODUCTION LESSONS LEARNED

SLOW DOWN



Speed is good, but stability is better.

VERIFY EVERYTHING



Check twice, execute once.

DOCUMENT CHANGES



Future you will thank you.

MONITOR ALWAYS



You can't fix what you don't see.

AUTOMATE REPEATED TASKS



Automation reduces human errors.

TEST DISASTER RECOVERY



A backup you haven't tested is not a backup.

LEARN & SHARE



Share knowledge, prevent others from falling.

PRODUCTION CHECKLIST BEFORE ANY CHANGE



- ✓ Do I understand the change?
- ✓ Have I tested in staging?
- ✓ Do I have a rollback plan?
- ✓ Have I taken a recent backup?
- ✓ Is this the right time to change?
- ✓ Have I informed the team?
- ✓ Will this impact users?
- ✓ Am I monitoring after the change?

REAL WORLD EXAMPLES

Mistake	Real World Impact
Run as root	Malware persistence, accidental deletion, privilege escalation
Wrong permissions	Application down, data leak
Delete critical files	System fails to boot
Ignore logs	Outage not detected until users complain
Poor backups	Data loss, long recovery time
No change control	Service disruption, config drift

RECOVERY MINDSET

- ✓ Stay calm
- ✓ Assess the impact
- ✓ Contain the issue
- ✓ Fix the root cause
- ✓ Restore service
- ✓ Document what happened
- ✓ Improve to prevent repeat



Resilient systems are built by learning, not by luck.

GOLDEN RULES FOR PRODUCTION

- ★ If you're not sure, don't do it.
- ★ If you didn't test it, don't deploy it.
- ★ If you can't rollback it, don't change it.
- ★ If you can't monitor it, you don't control it.
- ★ If you don't document it, it didn't happen.



REMEMBER

Production doesn't forgive mistakes, but it always rewards preparation, discipline, and continuous learning.



WHAT MAKES A GREAT ENGINEER?

- ✓ Takes responsibility
- ✓ Learns from mistakes
- ✓ Builds reliable systems
- ✗ Makes the next outage less likely





VERIQTA

20. VOLUME 1 REVIEW & MINI PROJECT

Practice makes perfect.
Build. Break. Fix.
Learn. Improve. Repeat.
You've got this!



MINI PROJECT: BUILD A LINUX SERVER

Hands-on project to apply everything from Volume 1!



- 1 Build Server**
Setup a fresh Linux server
- 2 Create Users**
Add users and groups
- 3 Configure Permissions**
Set correct ownership & permissions
- 4 Install NGINX**
Install and configure web server
- 5 Manage Services**
Manage and monitor services
- 6 Analyze Logs**
Read and analyze system logs
- 7 Monitor System**
Monitor CPU, memory, disk, network

PROJECT GOAL



Deploy a secure, monitored Linux server running NGINX, with proper users, permissions, logging, and monitoring.



Production Ready!

STEP-BY-STEP HANDS-ON PROJECT

- 1 Build a Linux Server**
 - Launch an Ubuntu 22.04 EC2 instance
 - Connect via SSH
- 2 Create Users**
 - Create 2 users: devops, deploy
 - Add deploy to sudo group
- 3 Configure Permissions**
 - Create /var/www/html
 - Set ownership to deploy
 - Set permissions 750
- 4 Install NGINX**
 - Install NGINX
 - Start and enable service
 - Configure firewall (UFW)
- 5 Manage Services**
 - Start, stop, restart NGINX
 - Enable on boot
 - Check status
- 6 Analyze Logs**
 - Read NGINX access & error logs
 - Use journalctl for service logs
- 7 Monitor System**
 - Check CPU, memory, disk, network
 - Use top, free, df, ss, iostat

```
ssh ubuntu@server_ip
```

```
sudo adduser devops
sudo adduser deploy
sudo usermod -aG sudo deploy
```

```
sudo mkdir -p /var/www/html
sudo chown -R deploy:deploy /var/www/html
sudo chmod -R 750 /var/www/html
```

```
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
sudo ufw allow 'Nginx Full'
```

```
sudo systemctl status nginx
sudo systemctl restart nginx
sudo systemctl enable nginx
```

```
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
sudo journalctl -u nginx -xe
```

```
top
free -h
df -h
ss -tulpen
```

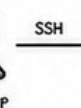
EXPECTED OUTCOME

- ✓ Server is running and reachable
- ✓ Users created with correct access
- ✓ NGINX is installed and serving
- ✓ Permissions are secure
- ✓ Services are managed and stable
- ✓ Logs are readable and monitored
- ✓ System resources are monitored



Your server is production ready!

PROJECT ARCHITECTURE



Your Laptop

Linux Server (Ubuntu)



PRODUCTION READINESS CHECKLIST

- ✓ System updated & patched
- ✓ Strong password / SSH key auth
- ✓ Firewall configured (UFW)
- ✓ Unnecessary services disabled
- ✓ Fail2ban installed & active
- ✓ Logs rotation configured (logrotate)
- ✓ Backups strategy in place
- ✓ Services enabled and monitored
- ✓ Disk space monitoring
- ✓ CPU & Memory monitoring
- ✓ Alerting setup (optional)
- ✓ Documentation created
- ✓ Access least privilege
- ✓ Tested & validated



SYSTEM MONITORING CHEAT SHEET

Area	Command	Purpose
CPU	top / htop	Real-time CPU usage
CPU (Detail)	mpstat -P ALL 1	Per-core CPU stats
Memory	free -h	Memory usage
Memory (Detail)	vmstat 1	Memory, CPU, IO stats
Disk	df -h	Disk usage
Disk (Detail)	du -sh *	Folder size
Disk IO	iostat -xz 1	Disk IO stats
Network	ss -tulpen	Network connections
Network (Detail)	ip -s link	Interface stats

TIP
Monitor consistently, alert early, solve faster!

INTERVIEW QUESTIONS (VOLUME 1)



- 1 What is the difference between df and du?
- 2 How do you check which process is using the most CPU?
- 3 How do you add a new user and set a password?
- 4 What is the purpose of chmod 755?
- 5 How do you check if a service is running?
- 6 How do you view logs for a service?
- 7 How do you troubleshoot high CPU usage?
- 8 How do you check open ports on a server?
- 9 What is the purpose of /etc/fstab?
- 10 How do you secure SSH access on a server?

BONUS CHALLENGE

Deploy your server on a cloud provider and make it accessible via your domain!



KEEP PRACTICING

The more you practice, the more confident you become!
Stay consistent!



KEY TAKEAWAYS FROM VOLUME 1

- ✓ You can manage files, storage, memory, CPU, and network.
- ✓ You can secure and access servers safely.
- ✓ You can automate tasks with bash scripts.
- ✓ You can troubleshoot like a pro.
- ✓ You are now ready for production environments!



REMEMBER

A strong foundation in Linux makes you a powerful engineer.

Keep building. Keep learning.
Keep growing!

WHAT'S NEXT?



Volume 2:
Advanced Linux Administration

- ✓ Advanced Networking
- ✓ System Tuning
- ✓ Security Hardening
- ✓ Automation at Scale and more...